Low-Level Design (LLD)

“RAG-Based College/University FAQ Assistant using LangGraph with Human-in-the-Loop (HITL)”

Module-Level Design

The system is divided into multiple modules, each responsible for a specific functionality.

1. Document Processing Module
   This module is responsible for loading the PDF document and extracting text from it. It acts as the first step in the data pipeline.
2. Chunking Module
   The chunking module divides the extracted text into smaller segments. This improves retrieval efficiency and ensures better context matching.
3. Embedding Module
   This module converts text chunks into vector embeddings using a pre-trained embedding model. These embeddings represent the semantic meaning of the text.
4. Vector Storage Module
   The vector storage module stores embeddings in ChromaDB. It enables efficient similarity search for retrieving relevant information.
5. Retrieval Module
   This module retrieves relevant chunks from the vector database based on the user query. It uses similarity search techniques.
6. Query Processing Module
   The query processing module handles user input and prepares it for further processing in the system.
7. Graph Execution Module
   This module uses LangGraph to manage the workflow of the system. It controls the flow of data between different components.
8. HITL Module
   The Human-in-the-Loop module handles escalation when the system cannot confidently answer a query. It allows human intervention to improve response quality.

Data Structures

The system uses structured data formats to manage documents, chunks, embeddings, queries, and workflow states.

1. Document Representation
   The document is represented as raw extracted text along with metadata.

Example:

```
{
"doc_id": "001",
"text": "Full extracted text from PDF",
"metadata": {
"source": "college_support.pdf"
}
}
```

2. Chunk Format
   The document is divided into smaller chunks, each stored with an identifier.

Example:

```
{
"chunk_id": "c1",
"content": "Students must maintain 75% attendance...",
"metadata": {
"page": 1
}
}
```

3. Embedding Structure
   Each chunk is converted into a numerical vector representation.

Example:

```
{
"chunk_id": "c1",
```

"embedding": [0.12, -0.45, 0.33, ...]
}

4. Query-Response Schema

   User queries and system responses are structured for processing.

Example:

{
"query": "What is minimum attendance?",
"retrieved_chunks": ["c1", "c3"],
"response": "Students must maintain 75% attendance.",
"confidence": 0.85
}

5. Graph State Object

   The state object is used in LangGraph to pass data between nodes.

Example:

{
"query": "...",
"context": "...",
"response": "...",
"confidence": 0.8,
"route": "auto" or "hitl"
}

Workflow Design (LangGraph)

The system uses LangGraph to manage the execution flow using nodes, edges, and a shared state object.

1. Nodes
   The workflow consists of the following nodes:

- Input Node
  Receives the user query and initializes the state.
- Processing Node
  Handles query processing, retrieves relevant chunks from the vector database, and generates a response using the LLM.
- Output Node
  Returns the final response to the user.

2. Edges
   Edges define the flow between nodes:

Input Node → Processing Node → Output Node

Additionally, conditional edges are used for routing:

Processing Node → HITL (if required)

3. State Management
   A shared state object is maintained throughout the workflow. It contains:

- User query
- Retrieved context
- Generated response
- Confidence score
- Routing decision

4. Execution Flow

- The user query enters the Input Node
- The Processing Node retrieves relevant context and generates a response
- The routing logic evaluates the response
- If confidence is sufficient, the Output Node returns the answer
- If confidence is low, the query is routed to the HITL module

Conditional Routing Logic

The system uses conditional logic to determine whether a query should be handled automatically or escalated to a human.

1. Answer Generation Criteria
   If the retrieved context is relevant and the LLM generates a clear response, the system proceeds with automatic response generation.
2. Confidence Evaluation
   A confidence score is assigned to each response based on retrieval relevance and response quality. If the confidence score is above a predefined threshold, the response is sent to the user.
3. Escalation Criteria
   The query is escalated to the HITL module under the following conditions:

- Low confidence score
- No relevant chunks retrieved
- Ambiguous or incomplete response
- Complex or multi-part queries

4. Routing Decision
   Based on the evaluation, the system assigns a routing label:

- "auto" → Direct response to user
- "hitl" → Escalation to human

5. Flow Control
   If the routing decision is "auto", the response is sent directly to the user.
   If the routing decision is "hitl", the query is forwarded to the HITL module for human intervention.

Human-in-the-Loop (HITL) Design

The Human-in-the-Loop (HITL) module ensures that the system can handle complex or uncertain queries by involving human intervention.

1. Escalation Trigger
   The system triggers HITL when:

- The confidence score is below the threshold
- No relevant information is retrieved
- The query is complex or ambiguous

2. Human Intervention
   Once escalation is triggered, the query is forwarded to a human operator. The human reviews the query and provides an appropriate response.
3. Response Integration
   The human-provided response is integrated into the system and treated as the final answer.
4. Final Output
   The validated response is sent back to the user through the interface.
5. Benefits

- Improves response accuracy
- Builds user trust
- Handles edge cases effectively

6. Limitations

- Requires human availability
- May increase response time
- Not fully automated

API / Interface Design

The system provides a simple interface for user interaction.

1. Input Format
   The user submits a query in the following format:

```
{
"query": "What is the attendance requirement?"
}
```

2. Output Format

   The system returns a structured response:

```
{
"answer": "Students must maintain 75% attendance.",
"source": "college_support.pdf",
"confidence": 0.85,
"route": "auto"
}
```

3. Interaction Flow

- User submits query
- System processes query
- Retrieves relevant context
- Generates response
- Applies routing logic
- Returns final answer

4. HITL Response Format (if applicable)

```
{
"answer": "Please contact the administration for special cases.",
"source": "Human Response",
"confidence": 1.0,
"route": "hitl"
}
```

Error Handling

The system includes mechanisms to handle errors and ensure smooth operation.

1. Missing Data
   If the PDF cannot be loaded or is empty, the system returns an appropriate error message.
2. No Relevant Chunks Found
   If the retriever fails to find relevant information, the system triggers HITL escalation.
3. LLM Failure
   If the language model fails to generate a response, a fallback message is returned.
4. Invalid Input
   If the user provides an invalid or empty query, the system prompts for a valid input.
5. System Failure
   General system errors are logged, and a user-friendly error message is displayed.